

Manual Técnico

Manual

Implementación de una herramienta web de visualización de datos, que
utilice los resultados del modelo

typst

Índice

Laboratorio de Desarrollo Regional

Descripción general	3
Stack tecnológico	4
Backend	4
Frontend	5
Arquitectura	6
Estructura del repositorio	6
Requisitos previos	8
Configuración	9
Configuración Local	9
Configuración Inicial del Entorno	11
Actualización de Covariables	13
Administrativas	13
Consumo Electrico (En sus distintas categorías)	15
Geoespaciales	17
Opciones de Despliegue en GCP	18
Google Compute Engine (Máquina virtual)	18
Requerimientos de la Máquina Virtual	19
Google Kubernetes Engine (GKE)	19
Servicio Administrado por Apache Airflow (Google Cloud Composer)	20
Variables de entorno	20
Raíz (.env-dev)	20
Roles y permisos	22

Credenciales del Cliente de Google Sheets	23
1. Habilitar la API en Google Cloud	23
2. Crear Credenciales	23
Credenciales del Cliente de Google Earth Engine para Autenti- cación No interactiva (Service Account)	24

Descripción general

Se desarrolló una herramienta web de visualización de datos de la estimación anticipada del PIB con variables geoespaciales.

El sistema está constituido modularmente como un sistema de microservicios, los cuales se listan en la siguiente tabla:

Módulo	Descripción
ETL (Automatización y calendarización de consulta y carga de datos)	El módulo se encarga de calendarizar las consultas a APIs geoespaciales y administrativas, para posteriormente preprocesarlas y enviarlas a la capa de persistencia de datos.
Modelación y pronóstico	El servicio se encarga del entrenamiento y pronóstico de modelos lineales y de aprendizaje de máquina.
Persistencia de datos	Se usa Delta Lake como capa de almacenamiento. Delta Lake transforma ese almacenamiento económico y flexible en una base sólida para arquitecturas modernas como el Lakehouse. La característica principal usada para la construcción del servicio es la de control de versiones (Time Travel): Permite consultar o restaurar versiones anteriores de tus datos. Es ideal para auditorías, reproducir experimentos de machine learning o corregir errores. Nota : El servicio de almacenamiento se trabajó de forma local con RustFS durante la etapa de desarrollo de la aplicación. Se recomienda usar un servicio de almacenamiento en la nube para centralizar el acceso a los datos. Se sugiere usar AWS S3 o el servicio de almacenamiento de Google.

Módulo	Descripción
Dashboard de administración	El Dashboard de administración permite interactuar al usuario con la capa de persistencia de datos, especialmente para la carga de datos administrativos para los cuales no hay API disponible. El Dashboard permite al usuario ejecutar el pronóstico de los modelos una vez que se cuentan con los datos disponibles.
Dashboard para la visualización de resultados	El Dashboard presenta los resultados de la estimación del valor del Índice de Volumen Encadenado (IVE) desestacionalizado para el último trimestre disponible así como la descomposición departamental del PIB Corriente.

Stack tecnológico

Backend

Módulo	Tecnología	API	Propósito
ETL (Automatización y calendarización de consulta y carga de datos)	Apache Airflow.	APIs usadas: Cliente Google Earth Engine, Cliente de Google Sheets, FRED y Servicio BlackMarble VIIRS de NASA.	Calendarización de las consultas a APIs geoespaciales y administrativas
Modelación y pronóstico	Python (paquetes estándar de aprendizaje de	Servicio REST de Apache Airflow para la ejecu-	Entrenamiento y pronóstico de modelos lineales

Módulo	Tecnología	API	Propósito
	máquina y modelación estadística : sklearn, xgboost, statsmodels) y R (modelo subnacional)	ción del contenedor que entrena y pronostica.	y de aprendizaje de máquina
Persistencia de datos	Delta Lake y RustFS		Capa de persistencia de datos

Frontend

Módulo	Tecnología	API	Propósito
Dashboard de administración	Data Studio (Looker Studio) y Google Sheets	Service accounts de google para habilitar los clientes de Google Earth Engine y Google Sheets	Visualizar los resultados de los pronósticos de los modelos lineales y subnacional

Arquitectura

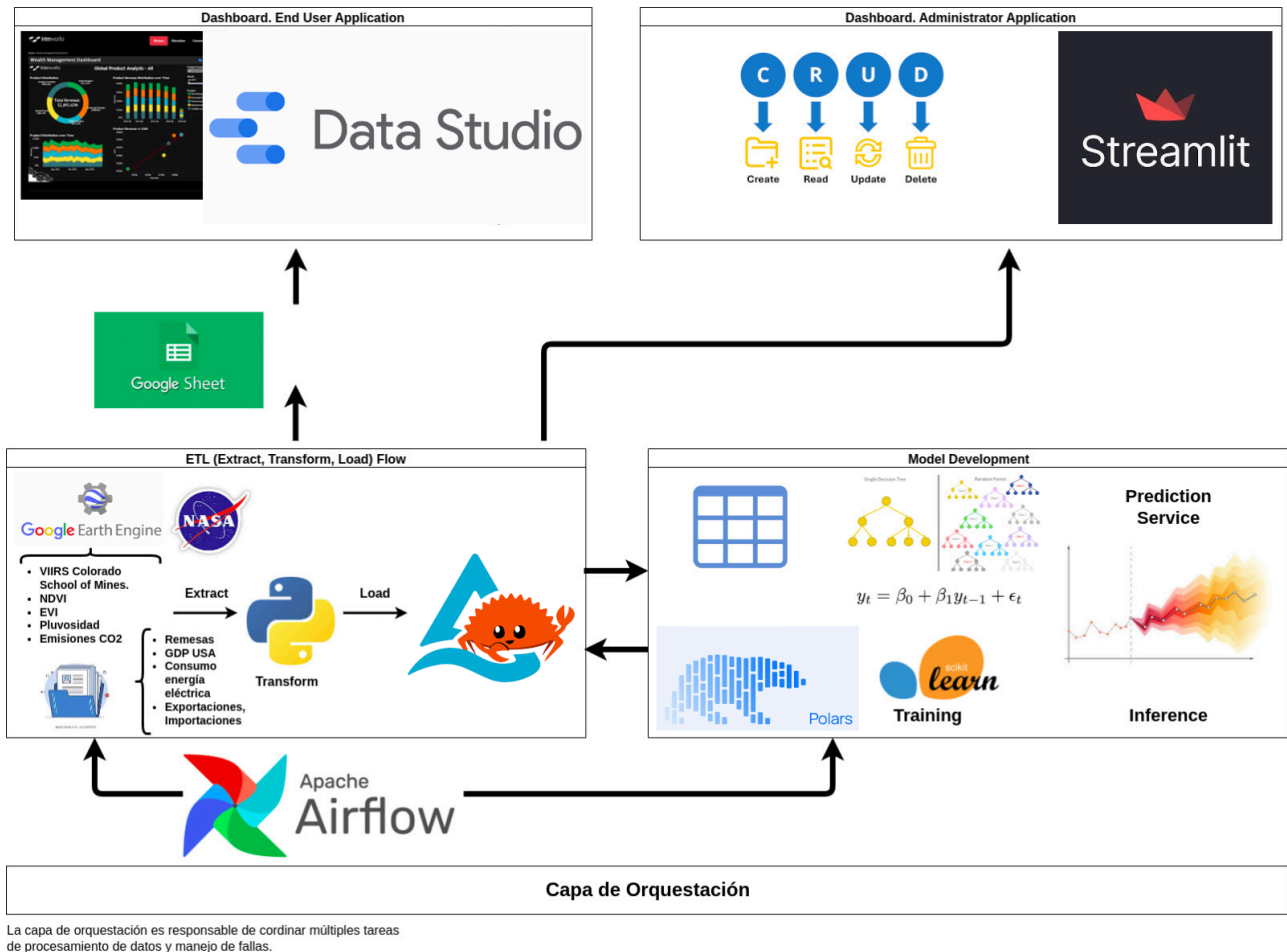


Figura 1: Arquitectura del Sistema

Estructura del repositorio

```

├── dags ## Directorio donde se encuentran las tareas calendarizadas
│   ├── crea_tablas.py
│   ├── pronostico_modelos_lineales.py
│   ├── pronosticos_modelos_ml.py
│   ├── pronostico_subnacional.py
│   ├── test.py
│   └── utils.py
├── docker-compose-dev.yml
├── docker-compose.yml
├── official-airflow-docker-compose.yml
├── pyproject.toml
├── README.md
├── services ## Directorio de los microservicios
│   ├── fetch_load_data ## Tareas de consultas a APIs geoespaciales y administrativas
│   │   ├── blackmarblepy_dist
│   │   │   └── blackmarblepy-2025.11.3.dev1+gc53615384.d20260516-py3-none-any.whl
│   │   ├── config
│   │   │   ├── general
│   │   │   └── config.toml

```

- └─ storage
 - └─ storage_config.toml
 - └─ Dockerfile
 - └─ environment.sh
 - └─ geojson
 - └─ gadm41_SLV_0.geojson
 - └─ gadm41_SLV_1.geojson
 - └─ pyproject.toml
 - └─ README.md
 - └─ tasks
 - └─ consumo_elect.py
 - └─ disaster_occurrence.py
 - └─ dmsp_csm_sum.py
 - └─ evi_gee.py
 - └─ export_usd_fob.py
 - └─ gdp_us_const_trim.py
 - └─ gee
 - └─ gee_functions.py
 - └─ import_usd_cif.py
 - └─ ndbi_gee.py
 - └─ ndvi_gee.py
 - └─ ndvi_wfp.py
 - └─ precip.py
 - └─ remesas_usd_trim.py
 - └─ temp_air.py
 - └─ temp_ls.py
 - └─ viirs_bm_departamentos.py
 - └─ viirs_bm.py
 - └─ utils
 - └─ __init__.py
 - └─ utils.py
 - └─ uv.lock
- └─ models ## Microservicio de Entrenamiento y Pronóstico de modelos lineales y de ML
 - └─ arimax_models.py
 - └─ arimax_models_rev.py
 - └─ crea_tablas.py
 - └─ Dockerfile
 - └─ Dockerfile-compose-dev
 - └─ environment.sh
 - └─ ml_models.py
 - └─ populate_pronosticos.py
 - └─ pyproject.toml
 - └─ README.md
 - └─ utils
 - └─ __init__.py
 - └─ utils.py
 - └─ uv.lock
 - └─ x13as
- └─ streamlit_app ## Microservicio de Dashboard de administración
 - └─ cov_administrativas
 - └─ administrativas.py
 - └─ __init__.py
 - └─ cov_geoespaciales
 - └─ geoespaciales.py
 - └─ __init__.py
 - └─ cov_subnacional
 - └─ __init__.py
 - └─ subnacional.py
 - └─ Dockerfile
 - └─ Dockerfile-compose-dev
 - └─ environment.sh
 - └─ metadata

```

├── admin
│   └── metadata_admin.toml
├── geoespacial
│   └── metadata_geoespacial.toml
├── subnacional
│   └── metadata_subnacional.toml
├── models
│   ├── __init__.py
│   └── models.py
├── pages
│   ├── actualiza_tokens.py
│   ├── covariables.py
│   ├── forecast_arimax.py
│   ├── forecast_ml.py
│   ├── forecast_subnacional.py
│   ├── __init__.py
│   ├── showcase.py
│   └── subnacional_covariables.py
├── pyproject.toml
├── README.md
├── streamlit_app.py
├── utils
│   ├── __init__.py
│   └── utils.py
├── subnacional ## Microservicio de Entrenamiento y Pronóstico del modelo subnacional
│   ├── bpvars.R
│   ├── crea_pdata_bpvar.py
│   ├── crea_tablas_raw.py
│   ├── datos
│   │   ├── electricidad_departamento.csv
│   │   ├── gdp_ppp_departamento.csv
│   │   ├── poblacion_departamento.csv
│   │   ├── raw
│   │   │   ├── panel_data_subnacional_forecast.xlsx
│   │   │   └── panel_data_subnacional.xlsx
│   │   └── viirs_bm_sum_departamento.csv
│   ├── Dockerfile
│   ├── Dockerfile-compose-dev
│   ├── envia_delta_lake.py
│   ├── envia_gs.py
│   ├── environment.sh
│   ├── pyproject.toml
│   ├── R_config.R
│   ├── README.md
│   ├── run_subnacional.sh
│   ├── utils
│   │   ├── __init__.py
│   │   └── utils.py
│   └── uv.lock
├── services-docker-compose-dev.yml
├── services-docker-compose.yml
└── uv.lock

```

Requisitos previos

- Instalación previa de Docker y Docker Compose.

- Preferentemente levantar los servicios en Linux. En caso que se use Windows, es necesario modificar el volumen de RustFS a un directorio permitido por Windows. Es necesario modificar la línea 12 del archivo `services-docker-compose-dev.yml` para definir una ruta adecuada para Windows **e.g** `C:\data:/data`.

Configuración

Configuración Local

- 1. Clonar el repositorio y entrar al directorio del proyecto.

```
git clone https://github.com/milocortes/pipeline_data_slv.git
```

Nota: En caso de no tener instalado Git, realizar la instalación con la instrucción:

```
sudo apt update  
sudo apt install git -y
```

- 2. Instalación de paquetería y configuración previa.

Ejecutamos el archivo `initial_config.sh`. El programa Docker y realiza la configuración de RustFS (crea directorios y asegura que el puerto 9000 esté abierto para accesos externos):

```
bash initial_config.sh
```

- 3. Ejecutamos el archivo `deploy_docker_compose_dev.sh`. El archivo corre las siguientes instrucciones:
 - Carga las variables de ambiente definidas en el archivo `.env-dev`
 - Añade permisos adicionales al archivo `/var/run/docker.sock`
 - Sobreescribe el archivo Docker compose base de Apache Airflow para agregar las rutas específicas de los microservicios así como para incorporar las variables de ambiente.
 - Ejecuta archivo de Docker compose.

```
bash deploy_docker_compose_dev.sh
```

Nota: Se asume que se ha realizado los pasos de postinstalación de Docker Engine en Linux para manejar Docker como un usuario no-root.

En caso contrario, seguir la siguientes instrucciones en esta [página](#).

- 4. Espere unos segundos y debería poder acceder a los servicios siguientes servicios:
 - Servicio de Apache Airflow
 - Servicio de Almacenamiento de RustFS
 - Dashboard de administración de Streamlit

Docker compose inicia servicios adicionales para el funcionamiento de Airflow, como se muestra en la siguiente figura:

```
[+] up 18/18
✓Image streamlit-dev           Built
✓Image models-dev:latest      Built
✓Image subnacional-dev:latest Built
✓Volume pipeline_data_slv_postgres-db-volume Created
✓Volume pipeline_data_slv_airflow-data-volume Created
✓Container streamlit_app       Started
✓Container pipeline_data_slv-postgres-1 Healthy
✓Container models              Started
✓Container pipeline_data_slv-redis-1 Healthy
✓Container rustfs-storage      Started
✓Container subnacional         Started
✓Container pipeline_data_slv-airflow-data-1 Started
✓Container pipeline_data_slv-airflow-init-1 Exited
✓Container pipeline_data_slv-airflow-apiserver-1 Healthy
✓Container pipeline_data_slv-airflow-dag-processor-1 Started
✓Container pipeline_data_slv-airflow-scheduler-1 Started
✓Container pipeline_data_slv-airflow-triggerer-1 Started
✓Container pipeline_data_slv-airflow-worker-1 Started
```

Figura 2: Contenedores ejecutados por Docker Compose

Este código expone los siguientes servicios en `localhost:[puerto]`, con `[nombre de usuario]/[contraseña]` indicados entre paréntesis:

- 8080-Aiflow (`airflow/airflow`)
- 9001-RustFS (`rustfsadmin/rustfsadmin`)
- 8502-Dashboard de administración de Streamlit (no requiere usuario ni contraseña)
- 5. Para detener los contenedores en ejecución, ejecute el siguiente comando:

```
bash stop_docker_compose_dev.sh
```

Configuración Inicial del Entorno

- 1. Entra al servicio de RustFS 0.0.0.0:9001 y crea el Bucket pronostico

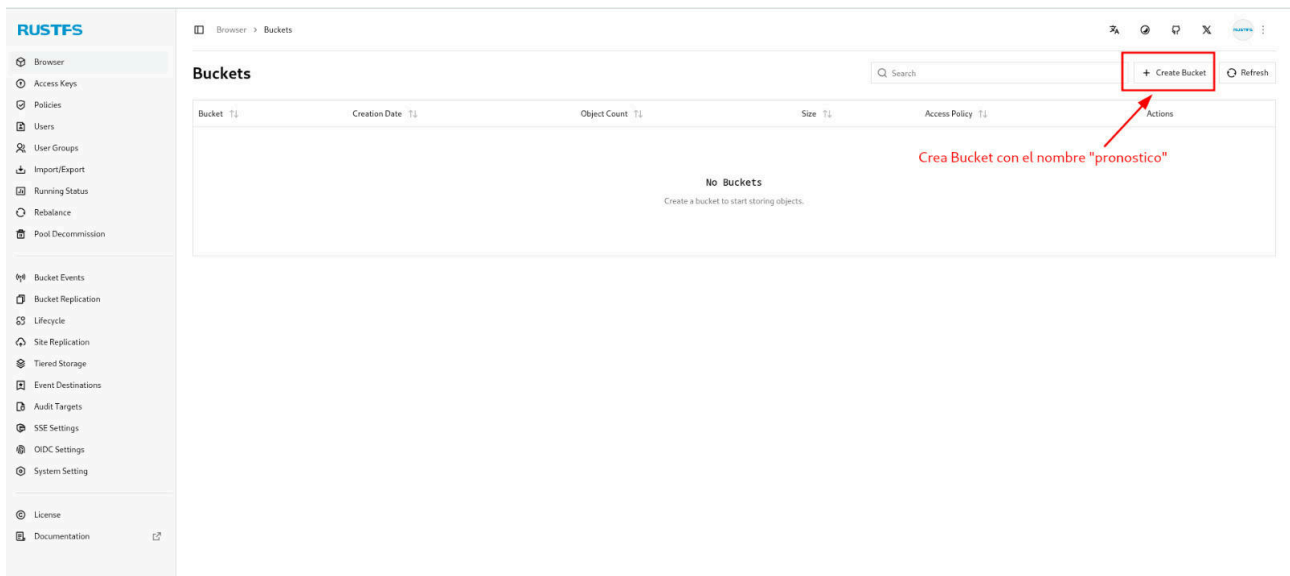


Figura 3: Creación de Bucket pronostico

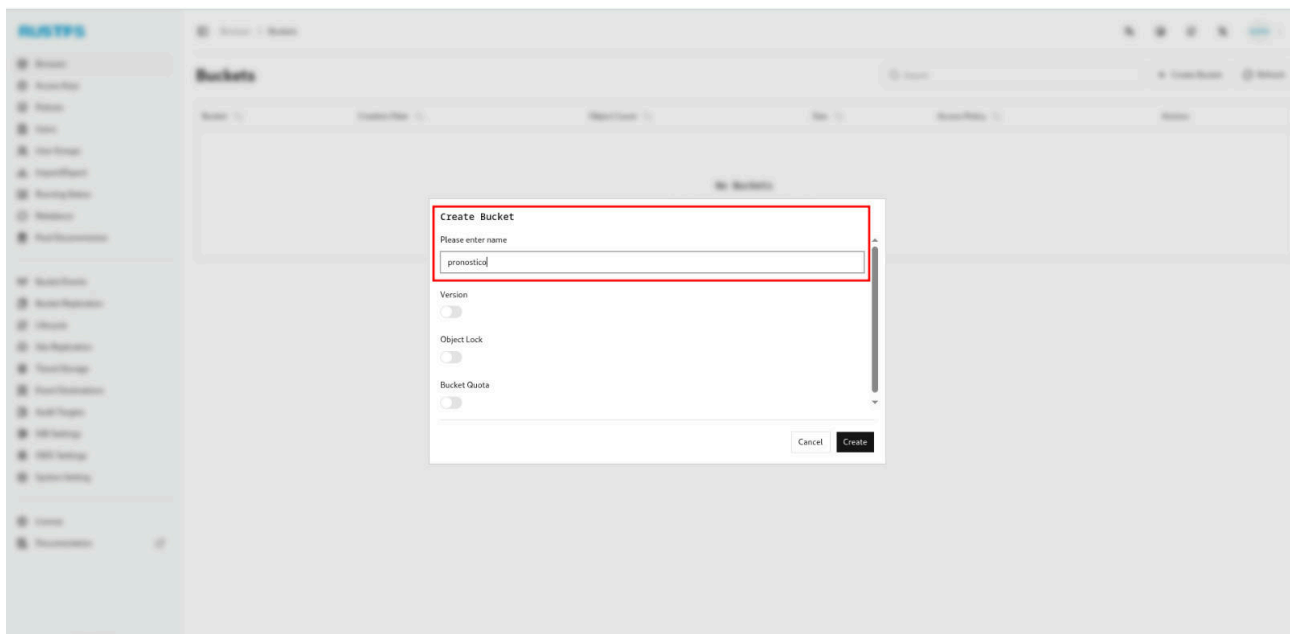


Figura 4: Creación de Bucket pronostico

- 2. Entra al servicio de Apache Airflow 0.0.0.0:8080 y ejecuta la DAG crea_tablas para inicializar las Tablas de covariables y pronósticos previos.

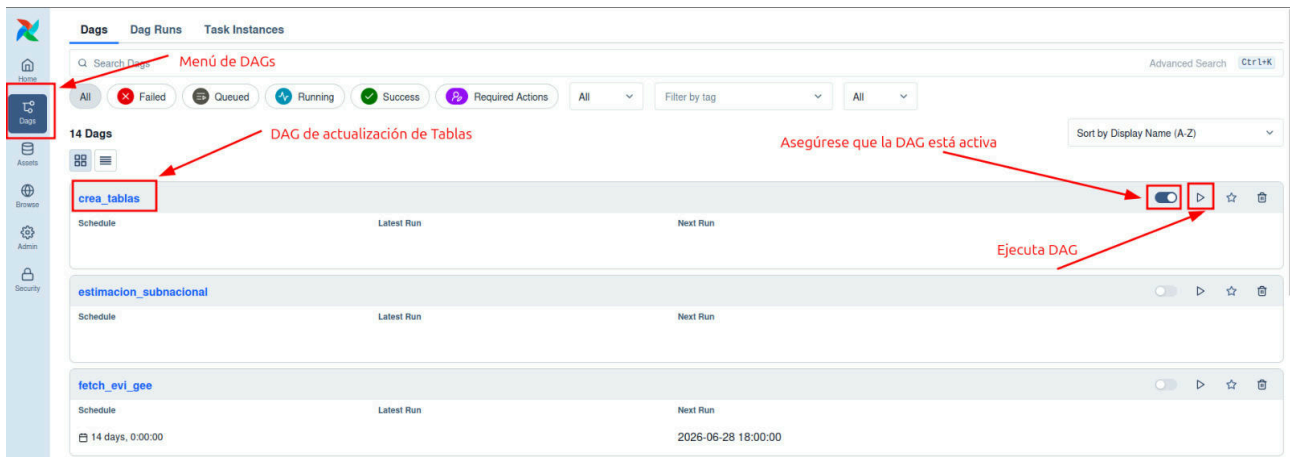


Figura 5: Creación de Tablas

- 3. Ir al Dashboard de administración de Streamlit y dar click en la opción de navegación de Actualización de Tokens. Es necesario actualizar los Tokens para los servicios de API de FRED y VIIRS BlackMarble de la NASA. Se espera que eventualmente el administrador del sistema genere sus propios tokens.

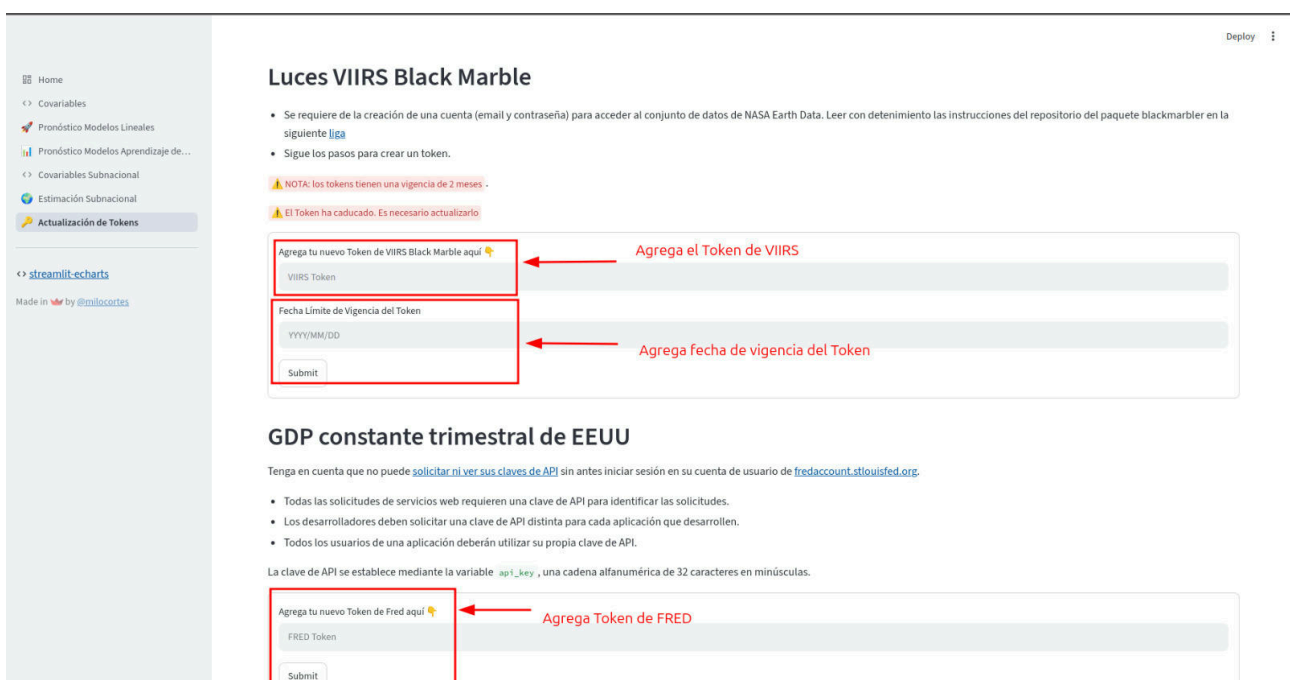


Figura 6: Actualización de Tokens

Por el momento, se comparten los Tokens usados para el desarrollo de la aplicación:

- FRED:
 - Token :
- VIIRS BlackMarble:
 - Token :
 - Fecha de Vigencia del Token: 3 de Septiembre de 2026.

Actualización de Covariables

Administrativas

No se cuenta con una API para la consulta de datos de las siguientes covariables:

- Índice de Volumen Encadenado
- Exportaciones
- Importaciones
- Remesas
- Consumo Eléctrico (En sus distintas categorías)
- Producto Interno Bruto Trimestral (PIB T). Producción y gasto a precios corrientes

En consecuencia, el proceso de actualización de información de estas variables debe ser manual mediante el acceso al sitio del Banco de la República (a excepción del Consumo Eléctrico), descarga manual del archivo en formato CSV (a excepción del Consumo Eléctrico el cual es compartido en formato Excel). En todos los casos (a excepción del Consumo Eléctrico), el usuario debe filtrar por el primer registro de 2012 hasta el último registro disponible y subirlo al sistema.

Las siguientes figuras muestran la configuración de descarga de cada una de las covariables descargadas del sitio del Banco de la República.

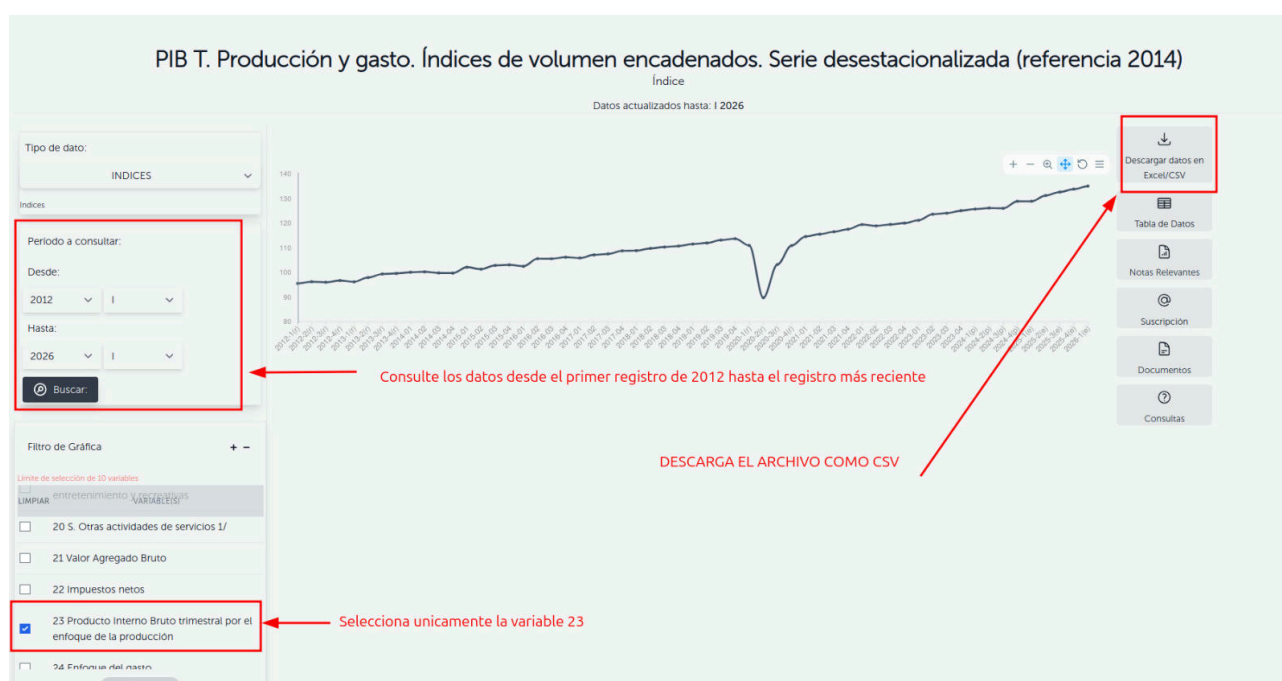


Figura 7: Índice de Volumen Encadenado

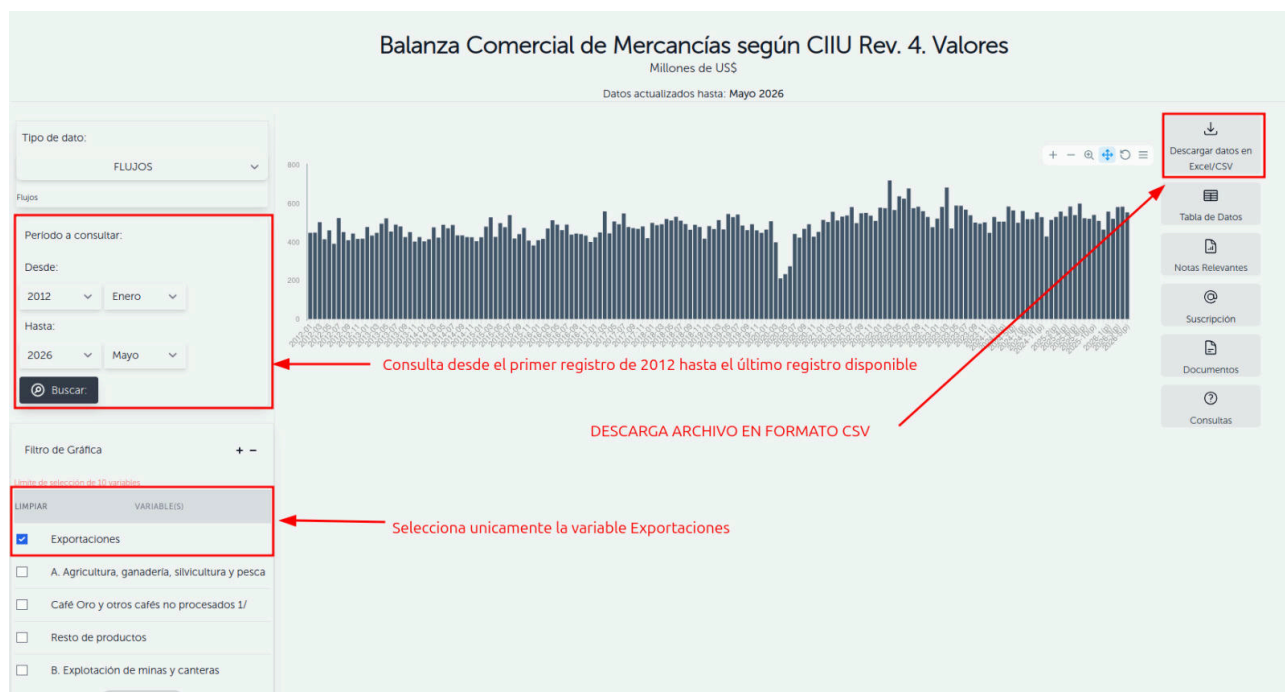


Figura 8: Exportaciones

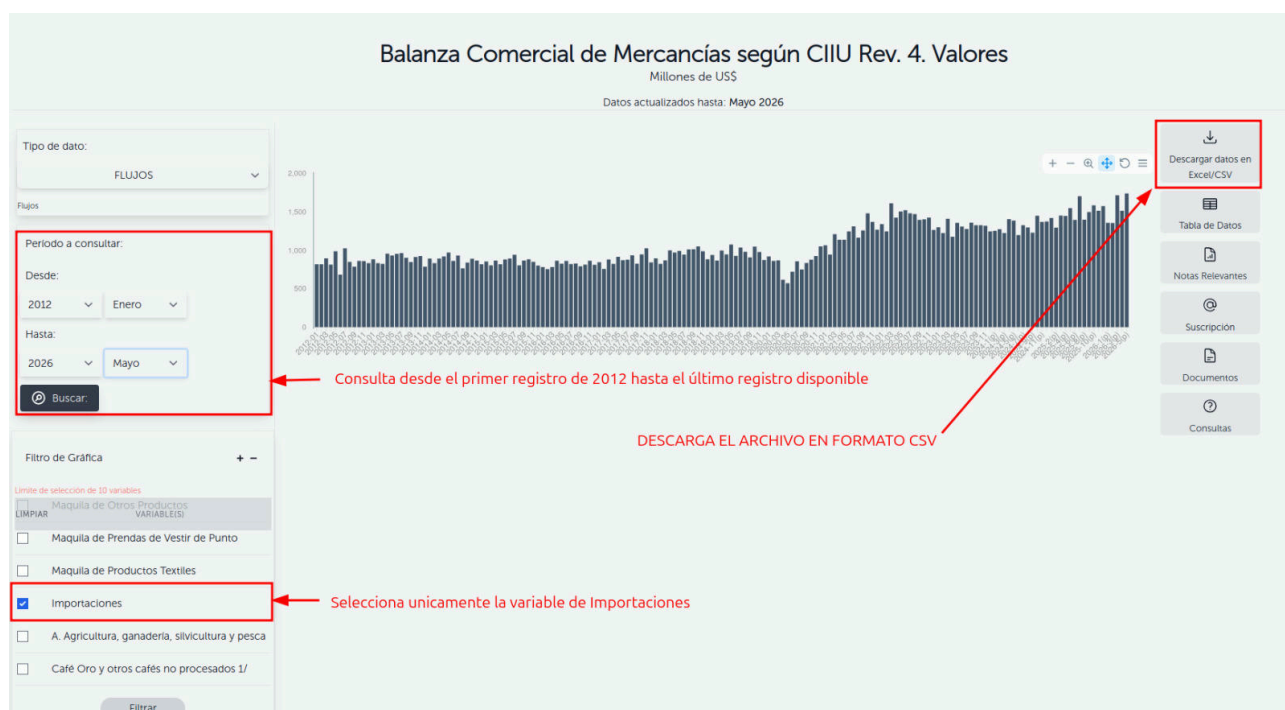


Figura 9: Importaciones



Figura 10: Remesas

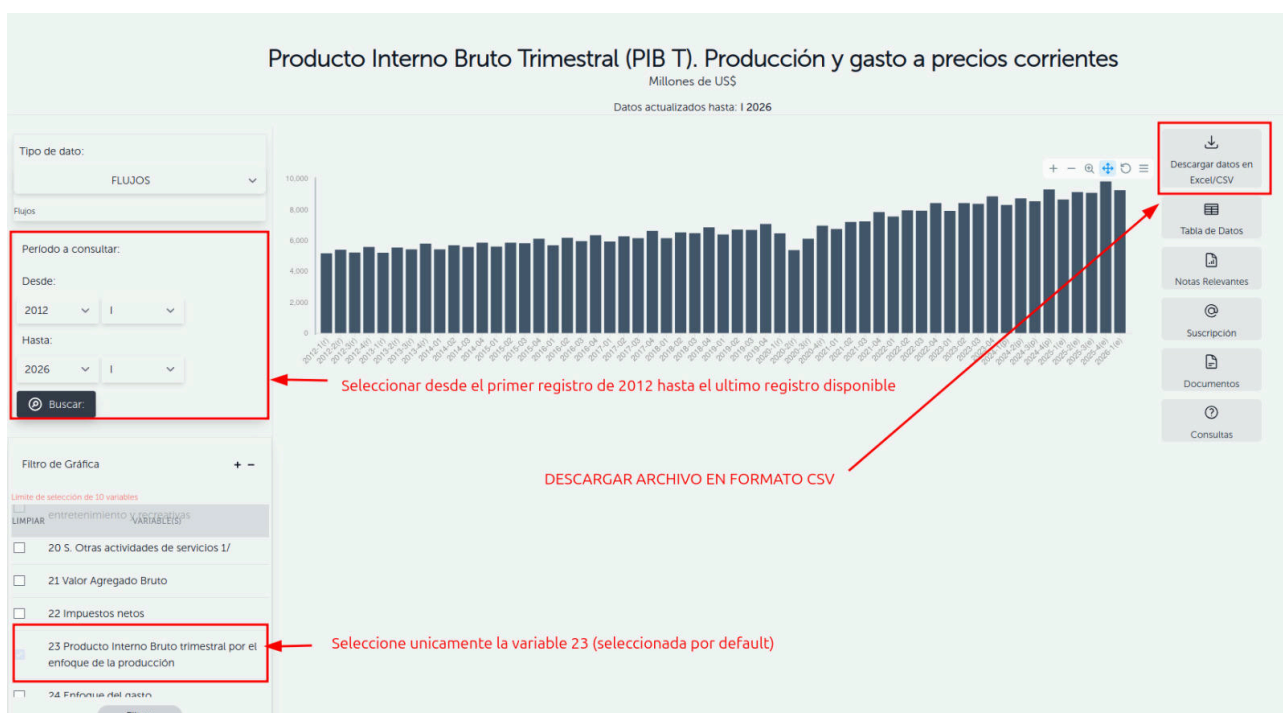


Figura 11: Producto Interno Bruto Trimestral (PIB T). Producción y gasto a precios corrientes

Consumo Electrico (En sus distintas categorías)

Los datos de **Consumo Electrico (En sus distintas categorías)**, tanto **Nacional** como **Subnacional** son compartidos internamente en formato Excel.

Para el caso del **Consumo Electrico Nacional** los datos son compartidos en un formato relativamente estructurado, de manera que es posible subir el archivo con el formato original en excel y el sistema realiza el preprocesamiento necesario para cargarse en el formato adecuado.

Sin embargo, para los datos **Consumo Electrico Subnacional**, la tabla no tiene un formato estructurado y no hay consistencia con los nombres de los departamentos. Por tal motivo, una vez el usuario tenga la información, este **deberá realizar un preprocesamiento previo de los datos de consumo eléctrico subnacional** con la finalidad de evitar problemas no previstos en la plataforma para su adecuada carga en la base de datos.

El usuario deberá subir a la plataforma un archivo csv el cual contendrá la tabla con la información subnacional con las siguientes columnas:

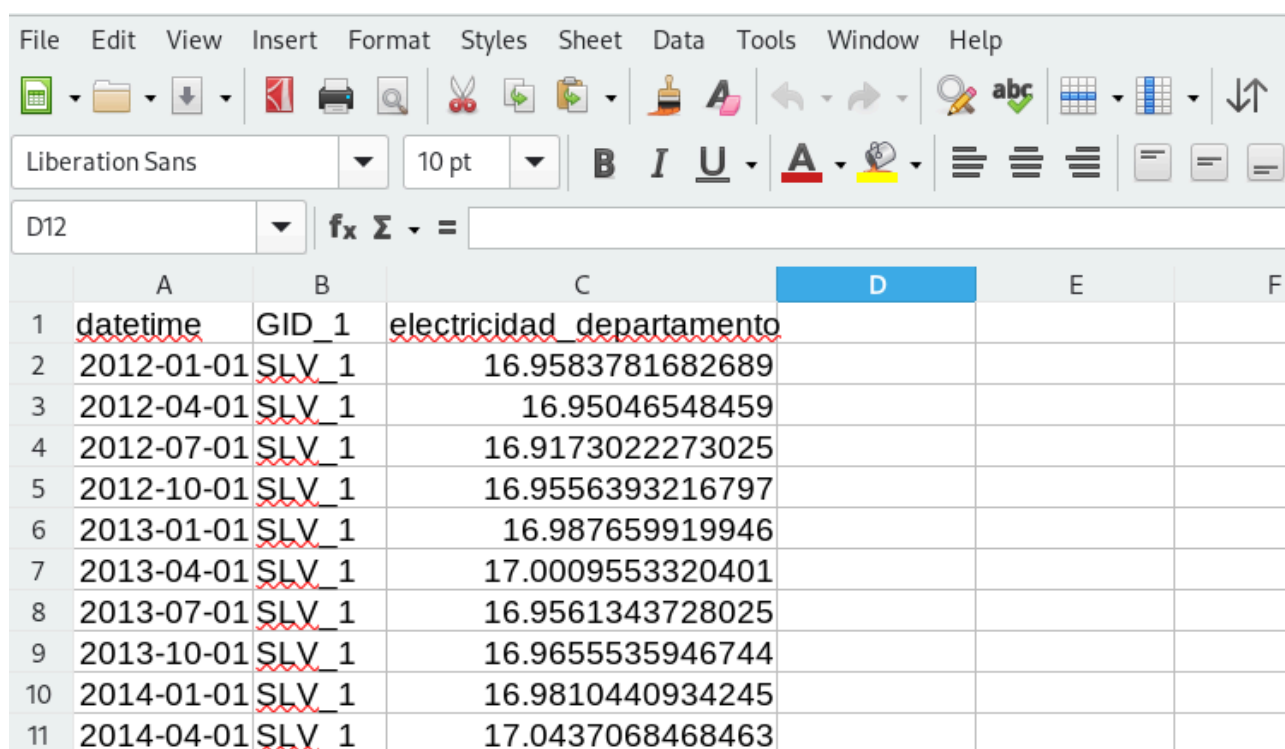
- `datetime` : Columna con tipo de dato cadena de **texto** el cual indica el trimestre. Los trimestres siguen el siguiente formato : [año]-01-01, [año]-04-01, [año]-07-01 y [año]-10-01 para hacer referencia al primer, segundo, tercero y cuarto trimestre, respectivamente.
- `GID_1` : Columna con tipo de dato cadena de **texto** que es un identificador alfanumérico único usado para para designar el límite administrativo subnacional principal de un país. Para los departamentos de El Salvador, contamos con los siguientes `GID_1` , los cuales son los valores posibles que puede tomar la columna `GID_1`:

GID_1	Departamento
SLV_1	Ahuachapán
SLV_2	Cabañas
SLV_3	Chalatenango
SLV_4	Cuscatlán
SLV_5	La Libertad
SLV_6	La Paz
SLV_7	La Unión
SLV_8	Morazán
SLV_9	San Miguel

GID_1	Departamento
SLV_10	San Salvador
SLV_11	San Vicente
SLV_12	Santa Ana
SLV_13	Sonsonate
SLV_14	Usulután

- `electricidad_departamento` : Columna con tipo de dato Flotante que contiene el consumo de energía eléctrica **en escala logarítmica**.

La siguiente figura muestra un ejemplo del formato que debe tener el archivo csv:



	A	B	C	D	E	F
1	<u>datetime</u>	<u>GID_1</u>	<u>electricidad_departamento</u>			
2	2012-01-01	SLV_1	16.9583781682689			
3	2012-04-01	SLV_1	16.95046548459			
4	2012-07-01	SLV_1	16.9173022273025			
5	2012-10-01	SLV_1	16.9556393216797			
6	2013-01-01	SLV_1	16.987659919946			
7	2013-04-01	SLV_1	17.0009553320401			
8	2013-07-01	SLV_1	16.9561343728025			
9	2013-10-01	SLV_1	16.9655535946744			
10	2014-01-01	SLV_1	16.9810440934245			
11	2014-04-01	SLV_1	17.0437068468463			

Figura 12: Formato del archivo csv del consumo de energía eléctrica subnacional

Geoespaciales

Las covariables geoespaciales se actualizan periódicamente mediante las APIs de Google Earth Engine y VIIRS BlackMarble de acuerdo a la política de calendarización definida en las DAGs de Apache Airflow.

Es necesario tener activas las DAGs de actualización de las covariables geoespaciales. Las DAGs correspondientes a estas tareas son las que inician con el prefijo fetch.

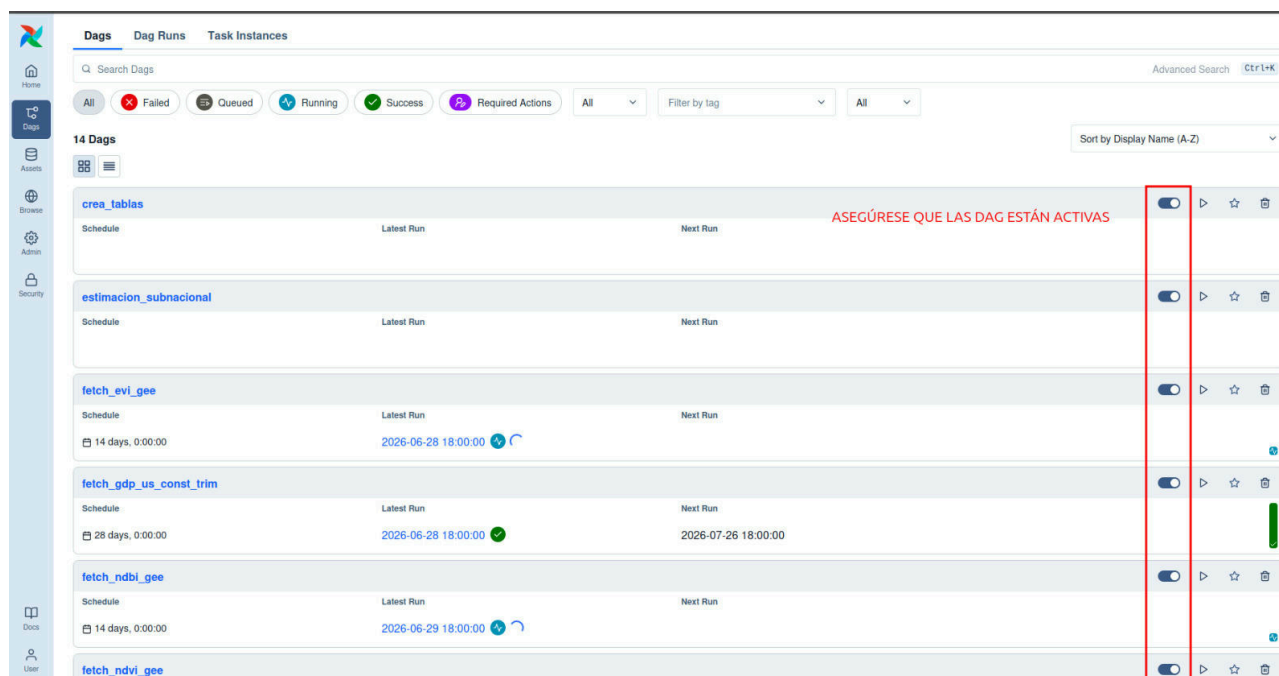


Figura 13: Activación de DAGs de actualización de covariables geoespaciales

Opciones de Despliegue en GCP

Google Compute Engine (Máquina virtual)

Una opción de despliegue de la aplicación es mediante su alojamiento dentro de una máquina virtual de Linux en GCP mediante el servicio Google Compute Engine. Los pasos del despliegue serían los siguientes:

- 1.- Crear máquina virtual con Google Compute Engine, ir a la consola de GCP, elegir una imagen estable de Linux (Debian o Ubuntu) y permitir el tráfico HTTP y HTTPS en los puertos requeridos por la aplicación (8080 y 8502).
- 2.- Conectarse via SSH a la máquina virtual.
- 3.- Instalar dependencias (git y docker)
- 4.- Descargar repositorio del programa, subir credenciales necesarias de los clientes de Google Earth Engine y de Google Sheets.
- 5.- Desplegar aplicación siguiendo los pasos de la [Configuración Local](#).
- 6.- Acceder a las Interfaces de Usuario en los puertos 8080 y 8502. Copiar la IP Pública de la máquina virtual y abrir `http://<YOUR_VM_EXTERNAL_IP>:{8080,8502}` en el navegador.

Nota: En el repositorio del sistema se encuentra la configuración de Terraform para levantar una instancia de Google Compute Engine.

En la configuración se levanta una instancia `e2-standard-4` con 2 CPU virtuales + 4 GB de memoria y una estimación mensual de USD24.46.

Para inicializar el directorio de trabajo local que contiene archivos de configuración de Terraform, se usa la instrucción :

```
terraform init
```

Para ejecutar las acciones propuestas en el plan de Terraform, usamos la instrucción (la cual solicitará un ID de proyecto)

```
terraform apply
```

Si deseamos eliminar permanentemente toda la infraestructura remota gestionada por su configuración actual de Terraform, usamos la instrucción :

```
terraform destroy
```

Requerimientos de la Máquina Virtual

El servicio de Apache Airflow consume muchos recursos. Se recomienda asignar a la máquina virtual al menos 8 CPU y 8 GB de RAM para el funcionamiento efectivo de la aplicación. La aplicación no es intensiva en almacenamiento de datos. Se recomienda asignar un almacenamiento de 500GB de espacio en disco.

Google Kubernetes Engine (GKE)

La opción de despliegue en Google Kubernetes Engine permite una mejor administración de la aplicación así como la integración con procesos CI/CD.

- 1.- Crear cluster de GKE. Dado que la aplicación no requiere alta disponibilidad, podemos crear un cluster pequeño con un pod donde se alojen los microservicios de la aplicación.
- 2.- Instalación de Helm. Helm es un administrador de paquetes de Kubernetes. Con Helm se instalará Apache Airflow. Es necesario configurar las imágenes de los microservicios para que los

servicio de Apache Airflow dentro sean alcanzables mediante la red interna del Cluster .

- 3.- Establecer una comunicación segura y temporal de los servicios de la máquina del usuario con el cluster de Kubernetes mediante `kubectl port-forward`. Este comando permite interactuar con bases de datos, APIs o Dashboards via localhost. Se recomienda esta forma de acceso para evitar exponer la aplicación al internet de forma pública. Un ejemplo del uso de `kubectl port-forward` es el siguiente:

```
kubectl port-forward <resource-type>/<resource-name> [local-port]:[target-port]
```

Servicio Administrado por Apache Airflow (Google Cloud Composer)

GCP cuenta con una versión de Apache Airflow lista para producción y completamente administrada por Google. GCP maneja el aprovisionamiento, escalamiento, parches de seguridad y alta disponibilidad de forma automática.

Esta es la opción menos recomendada dado el alto costo del servicio.

Variables de entorno

Raíz (.env-dev)

Variable	Uso
RUSTFS_ACCESS_KEY	Access Key del Servicio de almacenamiento de objetos RustFS
RUSTFS_SECRET_KEY	Secret Key del Servicio de almacenamiento de objetos RustFS
RUSTFS_CONSOLE_ENABLE	Habilita uso de la consola del Servicio de almacenamiento de objetos RustFS
RUSTFS_SERVER_DOMAINS	Nombre de dominio del Servicio de almacenamiento de objetos RustFS
AIRFLOW_UID	Identificador único de usuario asignado a Apache Airflow

Variable	Uso
AIRFLOW_GID	Identificador único de grupo asignado a Apache Airflow
_PIP_ADDITIONAL_REQUIREMENTS	Habilita proveedores de docker en Apache Airflow
AIRFLOW_IMAGE_NAME	Versión de la imagen de Apache Airflow
start_date	Fecha de inicio de covariables del sistema
LSIB	Dataset de Google Earth Engine con la geometría de delimitaciones nacionales
BUCKET_NAME	Nombre del bucket del Servicio de almacenamiento de objetos Rustfs
service_account_gee	Service account de Google Earth Engine
gee_project	Proyecto de Google Earth Engine usado para las consultas vía API
RUSTFS_DNS	DNS del servicio de RustFS
RUSTFS_PORT	Puerto de servicio de RustFS
AWS_REGION	Región de RustFS
AWS_ACCESS_KEY_ID	Access Key del Servicio de almacenamiento de objetos RustFS
AWS_SECRET_ACCESS_KEY	Secret Key del Servicio de almacenamiento de objetos RustFS
allow_http	Habilita el tráfico por http en RustFS
AIRFLOW_APISERVER_DOMAIN	Ruta del servicio REST de Apache Airflow

Variable	Uso
AIRFLOW_APISERVER_PORT	Puerto del servicio REST de Apache Airflow
_AIRFLOW_WWW_USER_PASSWORD	Contraseña del Usuario de Apache Airflow
_AIRFLOW_WWW_USER_USERNAME	Usuario administrador de Apache Airflow
GS_SPREADSHEET	Nombre del archivo de Google Sheets donde se encuentran los pronósticos
GS_API_KEY_FILE	Llaves para el acceso a la API del cliente de Google Sheets

Roles y permisos

Rol	Menú	Notas
Usuario Airflow	Dashboard de administración de Airflow	Agrega, elimina, y verifica estatus de DAGs de Airflow
Usuario RustFS	Dashboard de administración de RustFS	Crea, elimina y actualiza buckets y tablas en el almacenamiento de RustFS
Usuario Google Earth Engine	Dashboard de administración de RustFS	Habilita Cliente para la consulta via API de Google Earth Engine
Usuario Google Sheets	Dashboard de GS	Habilita Cliente para la actualización y eliminación en Google Sheets

Rol	Menú	Notas
Usuario Google Data Studio	Dashboard de Data Studio	Visualización y Administración del Dashboard de Data Studio

Credenciales del Cliente de Google Sheets

Para habilitar y utilizar la API de Google Sheets con Python, se debe configurar un proyecto en Google Cloud Console.

1. Habilitar la API en Google Cloud

1. Crea un proyecto: Ve a Google Cloud Console, haz clic en el menú desplegable de proyectos y selecciona «Proyecto nuevo».
2. Habilitar API: Vaya a API y servicios > Biblioteca. Busque y habilite tanto la API de Google Sheets como la API de Google Drive (Drive es necesario para el acceso completo a los archivos).
3. Configurar el consentimiento de OAuth: Ve a la pestaña de pantalla de consentimiento de OAuth, selecciona «Externo» (o «Interno» para Workspace) y proporciona el nombre de la aplicación y el correo electrónico requeridos.

2. Crear Credenciales

Elija alguno de los dos métodos de autenticación:

- OAuth 2.0 (aplicaciones de escritorio): Ve a Credenciales > Crear credenciales > ID de cliente de OAuth. Selecciona «Aplicación de escritorio», descarga el archivo JSON.
- Cuenta de servicio (scripts automatizados): Ve a Crear credenciales > Cuenta de servicio. Una vez creada, dirígete a la pestaña Claves, haz clic en Añadir clave > Crear nueva clave (JSON) y descárgala.
 - Paso crucial: Abre el archivo JSON, busca el `client_email` y comparte tu hoja de cálculo de Google con esa dirección de correo electrónico otorgando permisos de editor. **Aquí el administrador de sistema debe compartir el google sheets donde se guardan los resultados de los modelos con `client_email`.**

Credenciales del Cliente de Google Earth Engine para Autenticación No interactiva (Service Account)

Para flujos de trabajo automatizados, servidores o entornos de computación en la nube en los que no sea viable una solicitud de interacción en el navegador, debe utilizar una cuenta de servicio.

1. Crear una cuenta de servicio: En la consola de Google Cloud, crea una cuenta de servicio y genera un archivo de clave privada JSON.
2. Registrar la cuenta: Asegúrese de que el correo electrónico de la cuenta de servicio esté registrado para acceder a Earth Engine.
3. Utiliza el archivo de clave: en tu script de Python, emplea el siguiente código, sustituyendo `my-service-account@...gserviceaccount.com` y `.private-key.json` por tus propios datos: